

KVMI - Estado actual del proyecto

Relevamiento del codigo del repositorio: que esta implementado, que esta a medio camino y que sigue siendo un marcador de posicion, a la fecha de este corte.

Fecha de corte	1 de septiembre de 2026	Rama	main	Fuente	lectura directa del codigo, no de la documentacion previa
----------------	-------------------------	------	------	--------	---

01

Resumen ejecutivo

El producto que existe hoy en el codigo es una **casa de chocolate de lujo** con entrega en hoteles de Quito ("Deliver To My Hotel"), un ritual de degustacion guiado, un AI Concierge conversacional ya conectado, y un portal de Realidad Aumentada que todavia es un marcador visual. Esto se confirma en el prompt del concierge, el catalogo del frontend y los componentes de AR — no una galeria de arte, como todavia indica el titulo de `docs/ARCHITECTURE.md`.

El backend tiene **dos dominios contruidos en paralelo** que aun no se unificaron: uno mas simple y ya conectado al frontend (`catalog` , `orders` , `concierge`), y uno mas nuevo y mas completo pero todavia desconectado de la interfaz (`users` , `catalogue` , `commerce` , `logistics`). Las secciones siguientes documentan cada uno por separado.

Nota sobre la base de datos. El motor real, segun `settings.py` y el [CLAUDE.md](#) vigente, es **PostgreSQL** (con SQLite como salida de emergencia solo para desarrollo local). Comentarios sueltos en `CheckoutFlow.tsx` y en `docs/ARCHITECTURE.md` todavia mencionan SQL Server; quedaron desactualizados y conviene corregirlos.

Frontend - Astro, React, Tailwind

Version real instalada (`package.json`): Astro 7.2.2, React 19.2.8, Tailwind CSS 4.3.3 — por encima del piso nominal de CLAUDE.md (5.x / 18-19.x / 3.4+), sin conflicto.

Paginas implementadas

Home, El Origen, El Ritual KVMI, Collections, Exclusive Handicrafts, ficha de producto (`products/[slug]`), Checkout, Login y Contact, cada una con su version en `/es/` para el soporte bilingue (`src/i18n/ui.ts`).

Islas React e integracion con el backend

COMPONENTE	FUNCION	ESTADO
Concierge.tsx	Chat del AI Concierge	Conectado a <code>POST /api/concierge/</code>
ProductExperience.tsx	Ficha de producto interactiva	Implementado sobre datos locales
QuickCart.tsx	Carrito rapido	Implementado (localStorage)
CheckoutFlow.tsx	Bolsa -> Hotel Delivery -> Pago -> Confirmacion	Simulado - el comentario del propio archivo indica que el envio a <code>POST /api/orders/</code> es trabajo pendiente
AuthForm.tsx	Login / registro	Maqueta - sin llamada real; comentario: "Aqui se llamara a <code>POST /api/auth/</code> "

Carrito y catalogo en el cliente

`src/lib/cart.ts` mantiene el carrito en `localStorage` y sincroniza las islas via el evento `kvmi:cart`.
`src/lib/catalog.ts` define el catalogo completo (barras de chocolate, regalos romanticos, corporativos, premium, edicion limitada) de forma **local**, con imagenes propias — todavia no consume la API de Django.

Realidad Aumentada

`ARPortal.astro` es, segun su propio comentario, la "Estructura base (Fase 3)": un contenedor con un SVG de contingencia y el atributo `data-ar-portal` como futuro punto de montaje de WebXR o `model-viewer`. No hay logica AR real todavia.

Backend - dos dominios en paralelo

Dominio legacy (el que usa hoy el frontend)

CATALOG

Modelos: `Collection`, `Product`, `ProductImage`, `ProductIntent`. Endpoints de solo lectura (`GET /api/products/`, `GET /api/products/<slug>/`), funcionales.

ORDERS

Modelos: `Order`, `OrderItem`, `HotelDelivery`.
`POST /api/orders/` crea la orden completa dentro de una transaccion atomica, valida productos y campos de entrega. Implementado y funcional.

CONCIERGE

POST `/api/concierge/` conectado a la API de Gemini (`google-genai` , modelo `gemini-3.6-flash`). El catalogo que usa para recomendar son **3 piezas hardcodeadas en Python**, no la tabla `catalog_product` .

MIGRACIONES FALTANTES

`catalog` y `orders` solo tienen `__init__.py` en su carpeta de migraciones: nunca se genero `makemigrations` . Sus tablas no existen todavia en ninguna base de datos, pese a que son los apps que el frontend consume.

Dominio v2 (mas reciente, con migraciones, sin conectar al frontend)

Montado bajo el prefijo `/api/v2/` , para coexistir con el dominio legacy sin pisarlo.

USERS

`Customer` : identidad de cliente con campo `cdp_anonymous_id` preparado para un futuro CDP (Segment, segun el propio comentario). No tiene `urls.py` ni `views.py` : no hay endpoint de registro/autenticacion expuesto todavia.

CATALOGUE

`Product` con tipo de coleccion, perfil de sabor y `builder_content_id` (referencia a Builder.io como DXP). `ProductViewSet` de solo lectura, con filtros por coleccion y sabor.

COMMERCE

`Coupon` , `Cart` , `CartItem` , `Order` , `OrderItem` , `Payment` . El mas completo de los seis apps: agregar/editar items del carrito, aplicar cupon, y un endpoint `checkout` que calcula descuento, crea la orden y deja el `Payment` en `PENDING` , listo para conectar una pasarela.

LOGISTICS

`HotelDelivery` (con tipo y estado de entrega: reservado / en preparacion / entregado / confirmado) y `CrmLocalRecord` — buffer local para sincronizar con HubSpot (`hubspot_contact_id` , `lead_score` , `segment`). Ambos son solo el modelo de datos: la sincronizacion real con HubSpot no esta implementada.

Integraciones externas: estado real

Ninguna integracion de automatizacion, CDP o CRM externo esta conectada todavia. Lo que existe son **campos preparados** en los modelos, con comentarios explicitos de que la integracion se resolvera en una capa de servicios futura:

INTEGRACION PREVISTA	DONDE VIVE LA REFERENCIA	ESTADO
CDP (Segment)	<code>users.Customer.cdp_anonymous_id</code>	Campo placeholder
CRM (HubSpot)	<code>logistics.CrmLocalRecord.hubspot_contact_id</code>	Campo placeholder
DXP (Builder.io)	<code>catalogue.Product.builder_content_id</code>	Campo placeholder
Pasarela de pago	<code>commerce.Payment.gateway_provider</code>	Sin definir (nullable, a proposito)
Automatizacion (n8n)	-	No implementada - ningun webhook saliente todavia
AI Concierge	<code>concierge/views.py</code>	Conectado - Gemini (<code>gemini-3.6-flash</code>), no DeepSeek-V3

Tabla consolidada de avance

AREA	ESTADO	DETALLE
Paginas y navegacion (i18n es/en)	Implementado	9 rutas x 2 idiomas
Carrito de compra (cliente)	Implementado	localStorage, sincronizado entre islas
Catalogo (frontend)	Implementado	Datos locales en <code>catalog.ts</code> , no consume la API aun
Catalogo y ordenes (backend legacy)	Implementado sin migrar	Sin tablas creadas en base de datos
AI Concierge	Implementado	Gemini conectado; catalogo hardcodedo, no lee la BD
Checkout (envio real al backend)	Pendiente	Simulado en el cliente
Autenticacion de usuario	Pendiente	Maqueta sin backend
AR Experience	Placeholder visual	Sin WebXR/model-viewer real ("Fase 3")
Motor de comercio v2 (carrito/cupones/checkout)	Implementado, sin conectar	No consumido por el frontend todavia
Logistica Hotel Delivery v2 + CRM local	Implementado, sin conectar	Solo modelo de datos
Identidad de cliente (users)	Modelo sin API	Falta <code>urls.py</code> / <code>views.py</code>
CDP / CRM externo / DXP	No integrado	Solo campos preparados
Automatizacion (n8n)	No integrado	Sin webhooks
Pasarela de pago	No definida	<code>gateway_provider</code> nulo a proposito

Brechas a resolver antes de seguir avanzando

- **Duplicidad de dominio backend.** Conviven dos implementaciones de catalogo/ordenes (`catalog` / `orders` vs `catalogue` / `commerce` / `logistics`). Hace falta decidir cual es la definitiva y retirar o fusionar la otra, antes de que ambas sigan creciendo por separado.
- **Migraciones faltantes** en `catalog` y `orders` — justamente los apps que el frontend ya consume.
- **docs/ARCHITECTURE.md desactualizado:** el titulo dice "Digital Gallery" (no corresponde al dominio real de chocolate), menciona SQL Server en vez de PostgreSQL, y no cubre los apps v2 ni la conexion real con Gemini.
- **Checkout y autenticacion de cara al usuario** siguen siendo simulaciones de cliente sin conexion real al backend.
- **Entorno de desarrollo local:** `db.sqlite3` fue eliminado del repositorio sin que exista, en su lugar, una instancia de PostgreSQL documentada para levantar el proyecto.